Perfect 2048: the fewest moves, the highest score, the longest game, and what the undo button buys

Dom the Developer
https://github.com/DomTheDeveloper/2048-undo

Draft, September 2026

Abstract

We study 2048 under *possible play*: the spawn sequence is treated as part of the plan, which is exactly what a player with a re-randomising undo button can realise, and exactly what "possible with positive probability" means for the ordinary game. We prove five things. (1) On any board the tile 2^n cannot appear before move $(2^n - 8)/4 + (n - 2)$; the bound is attained (519 for the 2048 tile, 32,781 for the largest tile 131072 on 4×4), and every fastest 131072 passes through the same full board $65536, 32768, \ldots, 8, 4, 4$ at move 32,766 and folds it in fifteen forced merges. (2) On any board with c cells the score is at most $\Phi_c - 4c$, $\Phi_c = \sum_{k=2}^{c+1}(k-1)2^k$: reaching the top position costs one spawned 4 per tile, four points each; the proof is a register-allocation (Strahler number) argument. For 4×4 this is 3,932,100, so the circulating 3,932,164 and 3,932,156 are unattainable. (3) No game lasts more than $2^{c+1} - 4 - c$ moves (131,052 on 4×4), and the longest games are the highest-scoring games. (4) Every game that ends on the full chain $131072, \ldots, 4$ ends with two spawned 4s and the merge $4 + 4 \to 8$ from a full board, so the 4 always sits on the boundary beside the 8. (5) The rules are equivariant under the symmetries of the square and the eight corner snakes form one orbit, so one computed line is eight perfect games. Exhaustive enumeration confirms (1)–(4) on every board with at most nine cells, including the $2 \times 2 \times 2$ cube of Das and Paul's higher-dimensional game—the bounds are attained and every chain-producing move has the predicted form—and reproduces, to the digit, the 48,713,519 positions of 3×3 reported by Yamashita, Kaneko and Nakayashiki. For 4×4 we describe the algorithm that computed the shipped lines—a binary counter walked along the snake, its length fixed in advance by a mass ledger—and what is proved about its output: 32,781 moves to 131072 and 65,533 to the full chain, both optimal, and 129,333 moves for 3,925,224 points, 6,876 short of the ceiling, which we conjecture is attainable. The lines are published as plain-text witnesses (every slide, every spawned tile) with a 160-line verifier that depends on nothing but the rules; in particular, whether the standard game can reach 131072 at all, a question called open in 2017 and settled since only by an argument we could not verify, now has a certificate. Under honest 90/10 spawns the undo button must be pressed about 1.94 million times to play the 32,781-move line, a number we compute exactly. We also strongly solve the boards up to nine cells and benchmark an expectimax player against the classical heuristics on the full board.

1 Introduction

2048 [3] is played on a 4×4 board. A move slides all tiles in one of four directions; equal tiles that collide merge into their sum, each tile merging at most once per move; then one new tile appears

in a uniformly random empty cell, a 2 with probability 0.9 and a 4 with probability 0.1. The score is the sum of all merged values. The game ends when no move changes the board.

Alok Menghrajani's fork [22] adds an undo button that re-randomises the spawn on replay. That small change turns the game into a different object: a player who is willing to press undo often enough can realise *any* spawn sequence, and so can play any game that is possible in principle. This paper is about those games. Five questions organise it:

1. What is the fewest moves in which the largest tile can be built?
2. What is the highest score any game can reach?
3. What is the longest any game can last?
4. What must the final position of such a game look like, and in how many ways can it be arranged?
5. What does the undo button cost, in presses, to play such a game?

The first two questions have folklore answers. For the move count, the observation that every move adds a spawned 2 or 4 to the board gives a lower bound immediately; the cascade that finishes the tile adds a few more moves, and it was not obvious that the sum is exact. For the score, several numbers circulate: 3,932,164 (pure mass accounting), 3,932,156 (two "structurally forced" 4-spawns), and 3,932,100 (one forced 4-spawn per tile of the final position). Our own earlier README argued for a ceiling of 3,932,156 and claimed geometry prevents paying it. That claim was half right: geometry does prevent it, but the right ceiling is 3,932,100, and we prove it. Even the largest tile has a history: the tile-count argument that caps it at 131072 is old, but whether the spatial rules let a game actually reach it was doubted in 2014 [27] and still called open in 2017 [15]; Das and Paul [4] state it for boards of any shape, by an argument with a gap we discuss in Section 10. The line shipped with this paper is an explicit certificate for the standard game, of the minimum possible length, published as a text file that a short independent program checks (Section 6.4). The third and fourth questions we have not seen asked in print; their answers fall out of the same accounting.

Contributions.

- Theorem 4: 2^n needs at least $(2^n - 8)/4 + (n - 2)$ moves on any board, with equality for $n = 11$ (519) and $n = 17$ (32,781) on 4×4 and for every tile on every board of at most nine cells. Lemma 5: every fastest top tile passes through one and the same full board and finishes with a forced cascade.
- Theorem 9: on any c-cell board, score $\leq \Phi_c - 4c$. The argument identifies a merge tree's cell requirement with its Strahler number (the Ershov register count) and shows that the c final tiles must pay their tight moments in a forced order.
- Theorem 14: no game has more than $2^{c+1} - 4 - c$ moves, and the longest games are the maximum-score games up to their last spawn. Proposition 15: every game ending on the full chain ends the same way, which pins the 4 to the boundary beside the 8.
- Theorem 21: the rules commute with the eight symmetries of the square, the corner snakes form a single orbit, and one computed line is therefore eight perfect games—one per corner and direction.
- Exact values (Section 5.5): every bound above is attained on 2×2, 2×3, 2×4 and 3×3; the enumerator agrees with the published 3×3 state count.
- Section 6: the algorithm that computed the 4×4 lines and the certificate that makes its output a theorem (Theorem 19): 32,781 and 65,533 moves, optimal, and 129,333 moves for 3,925,224 points with 1,735 spawned 4s where the ceiling allows 16; we locate where the 4s go and conjecture the ceiling is attainable.

- Section 8: the exact expected number of undo presses for each shipped line under honest odds, and the opening problem.
- Section 9: a strong solution of the small boards (expected score, tile probabilities) and an expectimax player for the full board measured against the classical heuristics and an adversarial tile generator.

2 Related work

Complexity and combinatorics. Langerman and Uno [13] showed that deciding whether a given position can be played to a given tile is NP-hard on $m \times n$ boards for Threes, 1024 and 2048; Abdelkader, Acharya and Dasler [1] showed the variant without new tiles is still hard; Mehta [21] gave a PSPACE-completeness result for a related decision problem. Eppstein [5] analysed the maximum score in a relaxed model of 2048-like games in which any set of cells whose values sum to a tile value may be merged in one step, proving a min–max theorem that equates the maximum score with the smallest value for which greedy change-making needs a given number of coins; the binary-counter strategy every strong 2048 player follows is, in that light, greedy change-making in base 2. The relaxed model has only the small spawn value. The real game's 4-spawns are precisely the resource that buys an extra cell, and Theorem 9 prices that resource: four points each, and exactly c of them on the way to the top position.

Learning agents. Szubert and Jaśkowski [31] introduced temporal-difference learning of n-tuple networks for 2048; Wu et al. [33] added multi-stage learning; Jaśkowski [10] reached the strongest reported agents; Matsuzaki and co-authors studied the systematic selection of n-tuples and neural-network players [23, 18, 19, 20]; Guei, Wei and Wu [9, 8] used 2048-like games for teaching reinforcement learning and proposed optimistic TD learning. Slizkov [30] proved rigorous bounds on honest play: a strategy reaching 2048 with probability at least 0.99969, and 256 with certainty.

Exact solving. Lees-Miller [16, 14, 17] analysed the game with Markov chains and Markov decision processes, computed the minimum of 519 moves to the 2048 tile, and enumerated the 2×2 and 3×3 games. Yamashita, Kaneko and Nakayashiki [35] strongly solved 3×3 (48,713,519 positions) and Kaneko and Yamashita [11] strongly solved 4×3 (1,152,817,492,752 positions, 739,648,886,170 afterstates, optimal expected score about 50,724.26 from a two-2s start), the key technique being to partition the state space by the sum of tile values (the "age" of a state). Our enumerator uses the same grading and, on 3×3, reproduces their position count exactly (Section 5.5).

Higher dimensions. Das and Paul [4] analysed 2048 on $n_1 \times n_2$ boards and on d-dimensional boxes $n_1 \times \cdots \times n_d$: their Theorem 1 states that the largest tile 2^{c+1}, c the number of cells, can be reached on every such board, and they give strategies for the tile-placing adversary and, under conditions, for the player. Rodgers and Levine [27] had called 131072 the maximum attainable tile "in theory" while doubting the video evidence for it, and Kohler, Migler and Khosmood [12] repeat the 2^{n^2+1} figure. Our arguments use nothing of a board beyond its cell count and the fact that a slide acts line by line, so Theorems 4, 9 and 14 and Proposition 15 hold verbatim in every dimension, and Section 10 solves Das and Paul's $2 \times 2 \times 2$ cube exactly: every bound is attained there too.

Recent preprints. Saligram, Bhathal and Manihani [28] compare four deep reinforcement learning agents (DQN, PPO, QR-DQN and a distributional "Horizon-DQN"); the preprint reports the last reaching a 4096 tile with a best score of 41,828, and has since been withdrawn by its authors pending regenerated results. Bai, Kim Cohen, Koss and Lichtenbaum [2] evolve 2048 players with large language models, either by meta-prompting a "thinker"/"executor" pair, which did not improve, or by refining the value function of a limited Monte Carlo tree search, which gained about 473 points per cycle. An independent strong solution of the 3×3 board [25] reports an optimal expected score of 5,468.5 and tile probabilities of 99.571%, 73.677% and 1.134% for 256, 512 and 1024, which agree with Table 2 to every digit shown.

What is new here. The extremal questions of possible play do not seem to have been treated as theorems before. Lees-Miller's 519 is the length of a path in a Markov chain built for the 4×4 board by always taking the 4; Theorem 4 proves the formula behind it for every board and every tile, and Lemma 5 shows that every fastest top tile passes through the same board. Eppstein's min–max theorem is for a relaxed game with only 2-spawns and arbitrary set merges; in the real game the spawned 4s buy cells, and Theorem 9 prices them, which is why the answer is $\Phi_c - 4c$ and not a change-making number. The longest game (Theorem 14), the forced ending of every full-chain game (Proposition 15), the eight-spirals statement (Theorem 21), the exact undo expectations (Proposition 23) and the exact small-board values of Table 1 are, as far as we can tell, new; the reachability of the top tile, claimed in general by Das and Paul, gets an independently checkable certificate on 4×4 together with the exact fewest moves, and is joined by score and length ceilings that hold in any dimension, all attained on the first three-dimensional board solved; the maximum-score figures 3,932,164 and 3,932,156 that circulate are shown to be unattainable; and the 3×3 count of Yamashita et al. is independently reproduced. The snake strategy in one corner is folklore among players and the target of every heuristic program; that its eight images are all realised by one transformed move list is, to our knowledge, not stated anywhere, though it will surprise no implementer.

3 Preliminaries

A board is a finite set of c cells with a slide structure (rows and columns in the rectangular case; most arguments below use only the cell count). Tiles carry values 2^k, $k \geq 1$; we call k the *rank*. A *play* is a sequence of moves, each consisting of a slide (which performs its merges) followed by a *spawn* of a 2 or a 4 into an empty cell. The game starts with two spawns. Let n_2 and n_4 be the numbers of spawned 2s and 4s so far, counting the two starting tiles. A position is *full* when every cell holds a tile; on a full board a line (row or column) in which no merge is available cannot move.

Definition 1 (possible play)**.** In *possible play* the spawn values and cells are chosen by the player. Every finite spawn sequence has positive probability in the honest game, so a position is reachable in possible play if and only if it is reachable with positive probability in the honest game; and a player with a re-randomising undo button can realise any possible play.

For a position B write $\mathrm{mass}(B) = \sum_{t \in B} 2^{k_t}$ and

$$\Phi(B) = \sum_{t \in B} (k_t - 1)\, 2^{k_t}.$$

Lemma 2 (mass ledger)**.** *Slides conserve mass and every spawn adds its value:* $\mathrm{mass}(B) = 2n_2+4n_4$ *at all times. After* m *moves the mass is at most* $8+4m$*, with equality if and only if every spawn so far, the two starting tiles included, was a 4. Since every move spawns exactly once, the number of moves of any play reaching* B *is*

$$m = n_2 + n_4 - 2 = \tfrac{1}{2}\mathrm{mass}(B) - n_4 - 2.$$

Lemma 3 (score identity)**.** *At every moment of every play,* $\mathrm{score} = \Phi(B) - 4n_4$.

Proof. By induction over events. Initially the score is 0; a starting 2 contributes 0 to Φ, a starting 4 contributes 4 and raises $4n_4$ by 4. A merge of two 2^k tiles into 2^{k+1} scores 2^{k+1} and changes Φ by $k2^{k+1} - 2(k-1)2^k = 2^{k+1}$. A spawned 2 changes nothing on either side; a spawned 4 adds 4 to Φ and 4 to $4n_4$. □

So every spawned 4 costs exactly four points relative to what the final position "looks like it is worth", and the maximum score of a game is $\max_B\big(\Phi(B) - 4\,n_4^{\min}(B)\big)$ over reachable positions B, where $n_4^{\min}(B)$ is the fewest spawned 4s on any play reaching B. By the same token the *longest* play reaching B is the one with the fewest 4s: two plays to the same position with the same number of spawned 4s have the same length.

Merge trees. Every tile on the board is the root of a binary *merge tree*: its leaves are spawned tiles (value 2 or 4), each internal node is a tile created by merging its two children. A tile of rank k has all leaves at depth $k-1$ (2-leaves) or $k-2$ (4-leaves). Two facts about the game drive everything below:

- *Persistence.* A tile, once spawned or created, stays on the board until it merges into its parent. Nothing else removes it.
- *One doubling per move.* A tile's value at most doubles in a move, because each tile merges at most once per move.

Call the tiles present on the board that belong to the merge tree of a future tile T the *pieces* of T. By persistence, from the first spawn of a leaf of T until T is created, at least one piece of T is on the board; afterwards T itself is.

4 The fewest moves

Theorem 4. *On any board and for any play, the tile* 2^n $(n \geq 3)$ *does not exist before move*

$$M(n) = \frac{2^n - 8}{4} + (n-2).$$

Equality is attainable only if the game starts from two 4s and every spawn outside the last $n-2$ *moves is a 4.*

Proof. Let T be the first move after which a tile 2^n exists. A tile spawned at move $T-j$ has value at most $4 = 2^2$ then, and by one doubling per move its value at move T is at most 2^{2+j}; for it to be part of the 2^n we need $2 + j \geq n$, i.e. $j \geq n-2$. Hence the $n-2$ tiles spawned at moves $T-(n-3), \ldots, T$ are not part of the 2^n: they are still on the board (persistence), as themselves or merged among each other, so the position after move T has mass at least 2^n plus the sum of those $n-2$ spawn values.

Write z for the number of spawned 2s among the T move-spawns and z' for the number among the last $n-2$ of them, so $z' \le z$. By Lemma 2 the mass after move T is $M_0 + 4T - 2z$ with $M_0 \le 8$ the starting mass, and the last $n-2$ spawns sum to $4(n-2) - 2z'$. Therefore

$$\begin{aligned} M_0 + 4T - 2z \;&\ge\; 2^n + 4(n-2) - 2z', \qquad \text{so} \\ 4T \;&\ge\; 2^n - 8 + 4(n-2) + 2(z - z') + (8 - M_0) \;\ge\; 2^n - 8 + 4(n-2), \end{aligned}$$

which is the claim. Equality forces $M_0 = 8$ and $z = z'$. □

Attainability is a construction. For $n = 11$, Lees-Miller's Markov chain [16] exhibits the $519 = 510 + 9$ move win on 4×4. For $n = 17$ we generated, verified and ship a 32,781-move line (Section 6); Table 1 shows that the bound is attained for every tile on every board up to nine cells. For the largest tile a board can hold, the fastest plays are far more constrained than the theorem lets on: they all pass through one board.

Lemma 5 (the primed board)**.** *Let a play on a board with $c \ge 2$ cells create the tile 2^{c+1} at move $M(c+1)$. Then after move $M(c+1) - (c-1) = (2^{c+1} - 8)/4$ the board is full and holds exactly the tiles $2^c, 2^{c-1}, \ldots, 8, 4, 4$, every one of them a piece of the coming 2^{c+1}. In each of the remaining $c-1$ moves exactly one merge of these pieces takes place, creating $8, 16, \ldots, 2^{c+1}$ in this order, and none of the $c-1$ tiles spawned during those moves ever joins the 2^{c+1}.*

Proof. Put $n = c + 1$, $T = M(n)$ and $t_0 = T - (n-2)$. By the equality clause of Theorem 4 the play starts from two 4s and every spawn up to move t_0 is a 4, so after move t_0 the mass is $8 + 4t_0 = 2^n$. As in the proof of the theorem, a leaf of the merge tree of the 2^n spawned at move $T - j$ needs $j \ge n - 2$, so every leaf is spawned at or before move t_0. After move t_0 each leaf is contained in exactly one piece on the board (persistence), so the pieces present have total mass 2^n—the mass of the whole board. Every tile is therefore a piece, and every tile, being built from 4s, is a power of two of value at least 4.

The tile spawned at move t_0 is a 4 that must double in each of the moves $t_0 + 1, \ldots, T$. The remaining tiles are powers of two, each at least 4, summing to $2^n - 4 = 4\,(2^{n-2} - 1)$; in units of 4 they are powers of two summing to $2^{n-2} - 1$, whose binary expansion has $n-2$ ones, and a sum of powers of two has at least as many terms as the binary expansion of its value has ones. So there are at least $n - 2 = c - 1$ of them, at least c tiles in all on c cells: exactly c, the board is full, and equality in the binary bound means the $c-1$ remaining tiles are exactly $2^c, \ldots, 8, 4$.

At move $t_0 + 1$ the spawned 4 merges with the only other 4 into an 8; at move $t_0 + 2$ that 8 must double again, and the only other 8 is the primed one; inductively the k-th of the last $c-1$ moves merges the new 2^{k+1} with the primed 2^{k+1}. No tile spawned after t_0 can take part: after $k-1$ such spawns their total mass is at most $4(k-1) < 2^{k+1}$. All other primed tiles are distinct singletons, so no other merge among pieces is possible. □

For 4×4 the lemma says that every 32,781-move win shows, after move 32,766, the full board $65536, 32768, \ldots, 8, 4, 4$ (Figure 1, right), and then plays a cascade of fifteen forced merges $4 + 4$, $8 + 8$, …, $65536 + 65536$. The lemma leaves the *arrangement* free; the fifteen junk spawns of the cascade are the only freedom in the values. This is exactly what the "proven-minimal 15-move fold" of Section 6 realises. For tiles below the top one the pieces may be split further—at move 510 of a 519-move win the board holds pieces of the coming 2048 of total mass 2048, but they need not be $1024, \ldots, 8, 4, 4$: eleven tiles $1024, \ldots, 16, 4, 4, 4, 4$ can fold in the same nine moves by merging two pairs of 4s at once.

Corollary 6. *On a board with c cells the* full chain $2^{c+1}, 2^c, \ldots, 4$ *(one distinct power of two per cell, a dead position) needs at least $2^c - 3$ moves, and that many only if every spawn is a 4. On 4×4 this is* 65,533*, and it is attained.*

Proof. Its mass is $2^{c+2} - 4$; by Lemma 2 at least $(2^{c+2} - 12)/4$ moves are needed and every spawn must be a 4. The shipped line (Section 6) does it in exactly 65,533 moves; along it the 131072 first forms at move 32,784, three later than $M(17)$: all-4 feeding needs a few consolidations that a 2-spawn during the cascade would avoid, and the ledger absorbs them. □

5 The highest score, the longest game, and the last move

5.1 A merge tree needs its Strahler number of cells

The *Strahler number* of a binary tree is S(leaf) = 1 and, for a node with subtrees A, B, $\mathrm{S} = \max(\mathrm{S}A, \mathrm{S}B)$ if they differ and $\mathrm{S}A + 1$ if they are equal. It is the number of registers needed to evaluate an expression tree without spilling (Ershov [6], Sethi–Ullman [29], Flajolet–Raoult–Vuillemin [7]) and the black pebbling number of the tree [24]. Building a tile is evaluating its merge tree with cells as registers, and the persistence of pieces makes the lower bound short to prove directly.

Lemma 7. *Let T be a tile of rank $k \geq 2$ with merge tree $\mathcal{T}$. Then $\mathrm{S}(\mathcal{T}) = k$ if every leaf is a 2, and $\mathrm{S}(\mathcal{T}) = k - 1$ if at least one leaf is a 4.*

Proof. Induction on k. For $k = 2$: a spawned 4 is a leaf ($\mathrm{S} = 1$), and $2 + 2$ has $\mathrm{S} = 2$. For $k > 2$ the children are trees of rank $k - 1$ with Strahler numbers in $\{k - 2, k - 1\}$, equal to $k - 1$ exactly when all their leaves are 2s. Two all-2 children give $\mathrm{S} = k$; otherwise the maximum is $k - 1$ and, if both are $k - 2$, $\mathrm{S} = k - 1$ as well. □

Lemma 8 (tight moment)**.** *During the construction of a tile with merge tree $\mathcal{T}$ there is a moment at which at least $\mathrm{S}(\mathcal{T})$ pieces of $\mathcal{T}$ are on the board simultaneously.*

Proof. Induction on $\mathcal{T}$; a leaf is its own piece. Let the root have subtrees A and B, roots created at times $t_A < t_B$ (if they are created in the same move, either order works), merged at time $t^* > t_B$. Let τ_A, τ_B be tight moments of A, B given by induction, so $\tau_A \leq t_A$ and $\tau_B \leq t_B$. If $\mathrm{S}A > \mathrm{S}B$, or $\mathrm{S}B > \mathrm{S}A$, the larger side's tight moment already shows $\max(\mathrm{S}A, \mathrm{S}B)$ pieces. Suppose $\mathrm{S}A = \mathrm{S}B = s$; we need $s + 1$.

- If $\tau_B > t_A$: at τ_B the s pieces of B are on the board and so is the finished root of A (created at $t_A < \tau_B < t_B \leq t^*$, and persistent): $s + 1$ pieces.
- If $\tau_B \leq t_A$ and $\tau_A > t_B$: symmetric, s pieces of A plus the root of B.
- If $\tau_B \leq t_A$ and $\tau_A \leq t_B$: from τ_B until t_B at least one piece of B is on the board (persistence), and $\tau_A \leq t_A < t_B$; if $\tau_A \geq \tau_B$ then at τ_A there are s pieces of A and at least one of B. If instead $\tau_A < \tau_B \leq t_A$, then at τ_B there are s pieces of B and at least one piece of A (present from τ_A until $t_A \geq \tau_B$).

In every case some moment shows $s + 1$ pieces of $\mathcal{T}$. □

5.2 The ceiling

Let $\Phi_c = \sum_{k=2}^{c+1} (k - 1)2^k = (c - 1)2^{c+2} + 4$, the value of Φ on the full chain $2^{c+1}, 2^c, \ldots, 2^2$.

Theorem 9. *On any board with c cells, every play satisfies* score $\le \Phi_c - 4c$. *Equality requires the final position to be the full chain, reached with exactly one spawned 4 in the merge tree of each of its c tiles.*

Proof. Fix a play and its final position with tiles $T_1, \dots, T_m$, $m \le c$, of ranks $k_1, \dots, k_m$. Every spawned tile of the whole play is a leaf of exactly one T_i, so $n_4 = \sum_i h'_i$ where h'_i counts the 4-leaves of T_i; put $h_i = \min(h'_i, 1)$. By Lemma 7 the Strahler number of T_i's tree is $S_i = k_i - h_i$ (a tile of rank 1 is a spawned 2, $S_i = 1$).

Let τ_i be a tight moment of T_i (Lemma 8) and relabel so that $\tau_1 \le \tau_2 \le \cdots \le \tau_m$. At τ_p the board holds S_p pieces of T_p and, for each $q < p$, at least one piece of T_q or T_q itself (persistence, since $\tau_q \le \tau_p$). Hence $S_p + (p-1) \le c$, i.e.

$$k_p \;\le\; c + 1 - p + h_p \qquad (p = 1, \dots, m). \tag{1}$$

By Lemma 3,

$$\text{score} = \sum_p (k_p - 1)2^{k_p} - 4n_4 \;\le\; \sum_p \Big[(k_p - 1)2^{k_p} - 4h_p\Big].$$

For fixed p the bracket, subject to (1), is largest with $h_p = 1$ and $k_p = c + 2 - p$, worth $(c+1-p)2^{c+2-p} - 4$; the alternative $h_p = 0$, $k_p = c + 1 - p$ is worth $(c-p)2^{c+1-p}$, which is smaller for $p < c$ and equal ($= 0$) for $p = c$. All the terms are nonnegative, so filling the board ($m = c$) is best, and

$$\text{score} \;\le\; \sum_{p=1}^{c} \Big[(c+1-p)2^{c+2-p} - 4\Big] = \sum_{k=2}^{c+1} (k-1)2^k - 4c = \Phi_c - 4c.$$

Equality forces $k_p = c + 2 - p$ and $h_p = 1$ with $h'_p = 1$ for every p. □

Corollary 10. *On 4×4,* score $\le \Phi_{16} - 64 = 3{,}932{,}164 - 64 = 3{,}932{,}100$. *The numbers* 3,932,164 *and* 3,932,156 *are not attainable.*

Remark 11. The theorem uses only the number of cells, persistence, and one merge per tile per move. It holds for any adjacency structure, any spawn rule that offers the values 2 and 4, and in particular in possible play. The constraint that actually binds is that a tile of rank k built from 2s alone needs k cells at once, and the c tiles of the top position must take their tight moments in decreasing order of size on a board that the larger ones are already sitting on.

Inequality (1) is worth more than the score bound. It bounds the ranks of the final tiles by their tight-moment order, and so it says what the top tile and the top position are and what they cost.

Corollary 12 (the top tile)**.** *On a board with c cells no tile exceeds 2^{c+1}. Every tile 2^{c+1} has a spawned 4 among its leaves, and at some moment during its construction every cell of the board holds one of its pieces.*

Proof. A tile of rank k has Strahler number k or $k-1$ (Lemma 7) and needs that many cells at its tight moment (Lemma 8), so $k - 1 \le c$; for $k = c + 1$ the Strahler number must be c, which forces a 4-leaf, and the tight moment fills the board. □

That 131072 is the largest tile of 4×4 2048 is folklore, and Das and Paul [4] state the bound, and claim attainability, for boards of any shape and dimension; this is a two-line proof of the bound that says a little more: the 131072 is never built from 2s alone, and there is a moment when it owns the whole board.

Corollary 13 (the top position)**.** *On a board with c cells:*

(i) the full chain is the unique position of maximum mass, $2^{c+2}-4$;

(ii) every play reaching the full chain has at least c spawned 4s, one in the merge tree of each of its tiles; in particular its 4 is itself a spawned 4, and the chain cannot be built from 2s alone;

(iii) a play reaching the full chain has between 2^c-3 and $2^{c+1}-4-c$ moves.

Proof. By (1), $k_p \le c+2-p$, so mass $=\sum_p 2^{k_p} \le \sum_{p=1}^{c} 2^{c+2-p} = 2^{c+2}-4$ with equality only for the ranks $c+1, c, \dots, 2$: (i). For the chain $k_p = c+2-p$ forces $h_p = 1$ in (1): (ii). By Lemma 2 the number of moves is $\frac{1}{2}(2^{c+2}-4) - n_4 - 2$ with $c \le n_4 \le m+2$: (iii). □

5.3 The longest game

Theorem 14 (the longest game)**.** *On a board with c cells no play has more than $2^{c+1}-4-c$ moves. Equality requires the final position to be the full chain reached with exactly c spawned 4s, or the full chain with its 4 replaced by a 2 reached with exactly $c-1$ spawned 4s; in either case the score is $\Phi_c - 4c$. On 4×4 the bound is* 131,052*.*

Proof. By Lemma 2 the length is $\frac{1}{2}\text{mass} - n_4 - 2$. With the notation of the proof of Theorem 9, $n_4 \ge \sum_p h_p$ and (1) holds, so

$$\text{moves} \;\le\; \sum_{p=1}^{m} \bigl(2^{k_p-1} - h_p\bigr) - 2.$$

For $p < c$ the bracket is at most $2^{c+1-p}-1$ (take $h_p = 1$; the alternative $h_p = 0$ gives 2^{c-p}, which is smaller); for $p = c$ both choices give 1; and every bracket is positive, so $m = c$ is best. Summing, moves $\le \sum_{p=1}^{c}(2^{c+1-p}-1) - 2 = 2^{c+1}-4-c$. Equality forces $k_p = c+2-p$ and $h_p = 1$ for $p < c$, $(k_c, h_c) \in \{(2,1),(1,0)\}$, and $n_4 = \sum_p h_p$. The score follows from Lemma 3: the final 2 contributes nothing to Φ and saves one 4. □

So the longest game and the highest-scoring game are the same game up to its last spawn, and Conjecture 16 below is equivalent to the 4×4 longest game lasting 131,052 moves. The shipped maximum-score line, at 129,333 moves, is the longest 4×4 game we know of.

5.4 The last move

Proposition 15 (the last move)**.** *Let a play on a rectangular board with $c \ge 3$ cells end on the full chain after move L. Then:*

(i) the spawn of move L is a 4, and it is the 4 of the final position;

(ii) the slide of move L is played from a full board holding $2^{c+1}, \dots, 16$ and two 4s; it merges exactly one pair, $4+4 \to 8$, and moves nothing outside that line;

(iii) the spawn of move $L-1$ is also a 4;

(iv) the 4 of the final position occupies the end cell of the line of the last slide, opposite to the direction of the slide—a boundary cell—and the 8 lies in the same line.

Proof. Let F be the final multiset and s_t the value spawned at move t. The tile spawned at move L is on the final board as itself, so $s_L \in F \cap \{2,4\} = \{4\}$: (i). Before that spawn the board held $F \setminus \{4\}$, $c-1$ tiles. If the slide of move L merged j pairs, the position P after move $L-1$ held $c-1+j$ tiles, so $j \le 1$; and P contains the tile $s_{L-1} \in \{2,4\}$. If $j = 0$ then $P = F \setminus \{4\}$, which has no tile of value at most 4: impossible. So $j = 1$, P is full, and $P = F \setminus \{4, 2^k\} \cup \{2^{k-1}, 2^{k-1}\}$ for the

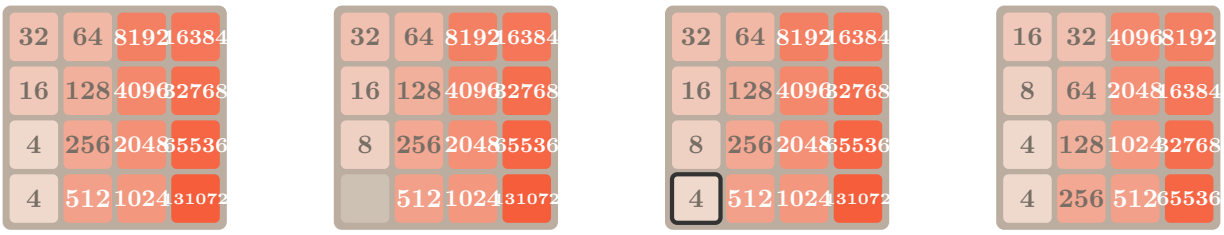


Figure 1: Left three boards: the last move of every full-chain game, here in the application's default orientation (bottom-right corner, chain running up). The board is full with two 4s; the slide (up) merges them into the 8 and frees the far end of the column; the final 4 spawns there. Right: the primed board of Lemma 5, the position after move 32,766 of every 32,781-move win, in the same arrangement.

merged pair; the only tile of P that can be at most 4 is 2^{k-1}, whence $k = 3$, $P = \{2^{c+1}, \ldots, 16, 4, 4\}$ and $s_{L-1} = 4$: (ii), (iii). On a full board a line without a merge cannot move; the merging line is full, so its two 4s are adjacent, and after the merge its three tiles pack toward the wall the slide moves to, leaving the far end cell empty—the only empty cell on the board, where the final 4 spawns: (iv). □

Proposition 15 is verified exhaustively on every board of Table 1 with at most nine cells—all 16, 112, 912 and 4,224 moves that produce the full chain on 2×2, 2×3, 2×4 and 3×3, and all 9,216 on the $2 \times 2 \times 2$ cube, have this form—and on the shipped lines, whose last three moves are $4 + 4 \to 8$, $8 + 8 \to 16$, $4 + 4 \to 8$ with four spawned 4s in a row (Figure 1). On 4×4 it says that in *every* reachable arrangement of the full chain the 4 is one of the twelve boundary cells and shares a row or column with the 8. The snake arrangements of Section 7 put the 4 at a corner with the 8 beside it.

5.5 Exact values on small boards

The mass of a position grows by exactly the spawned value each move, so the state graph is a directed acyclic graph graded by mass, and can be enumerated one layer at a time with only the two open layers in memory. Kaneko and Yamashita [11] call this grading the age of a state. We implemented it (`test/solve.js`) with positions reduced by the board's symmetry group and, per position, the fewest spawned 4s and the fewest moves on any play reaching it; the maximum score is then $\max_B(\Phi(B) - 4n_4^{\min}(B))$ by Lemma 3 and the longest game is $\max_B(\frac{1}{2}\mathrm{mass}(B) - n_4^{\min}(B) - 2)$ by Lemma 2. The same layered pass, run backwards, strongly solves the honest game: the optimal expected score and, for each tile, the maximum probability of ever building it.

Table 1 summarises. On every board tested the score ceiling is attained, always on the full chain, always with exactly one spawned 4 per tile, and the longest game is exactly $2^{c+1} - 4 - c$ moves and ends on the full chain or on the chain with a 2 in place of the 4, as Theorem 14 predicts; the fewest moves to every tile 2^k with $k \geq 3$ equals $M(k)$ of Theorem 4, and the fewest to the full chain equals $2^c - 3$. The 3×3 enumeration gives 48,713,519 positions, the number Yamashita, Kaneko and Nakayashiki report, which also fixes the convention of their count: positions up to the eight symmetries of the square. (Lees-Miller's "about 25 million" for 3×3 counts differently.) The honest-game side of the same computation is in Table 2.

board	c	positions	(raw)	max score	$\Phi_c - 4c$	$\Phi_c - 8$	chain in	longest	$2^{c+1} - 4 - c$
2×2	4	110	662	**180**	180	188	13	**24**	24
2×3	6	21,752	85,844	**1,260**	1,260	1,276	61	**118**	118
2×4	8	4,980,767	19,920,382	**7,140**	7,140	7,164	253	**500**	500
3×3	9	48,713,519	388,921,077	**16,352**	16,352	16,380	509	**1,011**	1,011
$2 \times 2 \times 2$	8	702,581	32,187,234	**7,140**	7,140	7,164	253	**500**	500
4×4	16	?	?	$\geq$ 3,925,224	3,932,100	3,932,156	65,533	$\geq$ 129,333	131,052

Table 1: Exact extremes of possible play. "positions" counts reachable positions up to the board's symmetries (8 for squares, 4 for rectangles, 48 for the cube), "raw" without reduction; "chain in" is the fewest moves to the full chain ($2^c - 3$, Corollary 6); "longest" the longest game (Theorem 14). On every board up to nine cells, the three-dimensional one included, the maximum score equals the ceiling of Theorem 9, the full chain is reached with exactly c spawned 4s, and the longest game equals the bound. The 3×3 count agrees with Yamashita et al. [35, 11]. The 4×4 row holds the bounds of this paper and the shipped lines.

board	$\mathbb{E}$[score], standard start	given a two-2s start	max probability of the largest tiles
2×2	66.96	67.70	16: 96.25%, 32: 8.29%
2×3	480.26	480.99	64: 93.93%, 128: 5.70%
2×4	2,641.87	2,642.60	256: 86.98%, 512: 2.46%
3×3	5,468.49	5,469.21	512: 73.68%, 1024: 1.13%
$2 \times 2 \times 2$	2,953.42	2,954.15	256: 94.04%, 512: 6.32%

Table 2: Strong solutions of the honest game (uniform empty cell, 90% twos). The expected score is under the score-maximising policy; each tile probability is under the policy that maximises that probability. The two-2s column uses Kaneko and Yamashita's convention (their 4×3 figure is 50,724.26). The 3×3 row agrees with the independent solution of [25]. The cube has six directions, and with the same eight cells is a better board than the 2×4 strip.

5.6 The 4×4 board

The 4×4 state space is out of reach of enumeration (Lees-Miller's month-long run [14] enumerated $1.3 \cdot 10^{12}$ states without finishing; the full game is far larger). What we have is a constructive lower bound. The generator of Section 6 walks a binary counter along the snake path into a corner, feeding 2s, and grants a spawned 4 only when a bounded look-ahead shows every pure-2 continuation dying. It produced a line of 129,333 moves ending on the full chain with $n_4 = 1{,}735$, hence a score of

$$\Phi_{16} - 4 \cdot 1{,}735 = 3{,}932{,}164 - 6{,}940 = 3{,}925{,}224,$$

99.825% of the ceiling and 6,876 points short of it. The identities $\text{moves} = 131{,}068 - n_4$ (all 129,333 moves spawn 2 except the 4s; mass $262{,}140 = 8 + 2 \cdot \text{moves} + 2n_4$) and $\text{score} = 3{,}932{,}164 - 4n_4$ are verified by replay through the game engine in all four corners and both spiral orientations.

Where do the 1,735 go? The first act—building 131072 on the empty board—spends exactly one, at its tight moment. The whole gap is in the second act, the chain $65536, \ldots, 4$ on the fifteen cells beside the 131072: 295 spawned 4s while building the 65536 and 1,439 below it, most of them with three to five empty cells still on the board. So the generator is not being squeezed by the pebbling bound; it is being squeezed by geometry within a cascade. When the counter overflows, the resulting chain of merges takes one move per level, a tile spawns during each of those moves, and the spawns land in the cells the cascade has just freed, alongside the tail of the chain. The

bounded look-ahead cannot always see how to digest that junk with 2s alone and buys its way out with a 4. Theorem 9 says the price of the second act is at least fifteen 4s; the small boards, which contain the same corner turns and the same overflow cascades in miniature, all pay exactly the theorem's price.

Conjecture 16. *The maximum score in* 4×4 *2048 is* 3,932,100*, attained by a play with exactly sixteen spawned 4s; equivalently (Theorem 14), the longest* 4×4 *game has* 131,052 *moves.*

Two routes to settle it suggest themselves. The direct route is a better search for the second act: the target is a schedule that, at each level k of the chain, builds the first copy of 2^{k-1} from 2s on the $k-1$ free cells (which Lemma 7 permits) and the second copy with a single 4 at its own tight moment, and that handles the overflow junk explicitly. The indirect route is Kaneko and Yamashita's enumeration: their layered pass over 4×3 can carry a two-byte "fewest 4s" field at negligible cost and would settle the 4×3 maximum, which Theorem 9 bounds by $\Phi_{12} - 48 = 180{,}180$, and the 4×3 longest game, bounded by $2^{13} - 16 = 8{,}176$; we predict equality in both.

6 The perfect algorithm

The lines the application plays were not found by searching the game at run time. They were computed once, offline, by a program that knows in advance how long its line must be, checked by an independent replay, and shipped as data (`js/perfect_line.js`, two hex characters per move: direction, spawn cell, spawn value); the browser replays them. This section describes the construction and, more to the point, what is *proved* about its output. The heuristics need no proof: they only decided whether a line was found. The theorems of the previous sections decide whether the line found is optimal.

6.1 The length is fixed before the search starts

Lemma 2 turns the two move-minimal targets into fixed-length problems. Under all-4 spawns the mass after m moves is $8 + 4m$, so the primed board of Lemma 5 (mass 131,072) is reached after exactly 32,766 moves on *every* route that reaches it at all, and the full chain (mass 262,140) after exactly 65,533. The tile count is pinned too: two tiles at the start, one more per spawn, one fewer per merge, sixteen on the primed board, so the build performs exactly $2 + 32{,}766 - 16 = 32{,}752$ merges in its 32,766 moves—a carry on all but fourteen of them. The generator therefore never optimises length. It has to find a route on which every spawn is a 4, and the ledger does the rest; the program asserts $8 + 4 \cdot \text{steps} = \text{mass}$ at every checkpoint and refuses any line that overruns the ledger. For the maximum-score line the roles reverse: spawns are 2s wherever possible, the only free quantity is n_4, and by Lemmas 2 and 3 $\text{moves} = 131{,}068 - n_4$ and $\text{score} = 3{,}932{,}164 - 4n_4$: the line is judged by one number.

6.2 The snake and the counter

Definition 17 (corner snake)**.** Fix a corner κ of the 4×4 board and one of the two sides through it. The *corner snake* $\sigma(\kappa, \text{side}) = (S_0, \ldots, S_{15})$ is the boustrophedon path that starts at κ, runs along the chosen side, and turns back at every wall: it visits the four lines parallel to that side in order, alternating direction. Figure 3 shows all eight. The *chain along* σ is the full chain arranged with 2^{17-i} on S_i (Figure 2).

The construction keeps the occupied prefix of the snake a binary counter: values non-increasing along $S_0, S_1, \ldots$, with at most one adjacent equal pair (a pending carry), and everything past the prefix small feed material. A spawn on the first free snake cell increments the counter, a slide toward the head of the line being filled performs a carry, and when a line fills, the carry turns the corner into the next line. The evaluation is

$$H(B) = \sum_{i<\ell} v_i \, 2^{-i} \;-\; 10^3 \sum_{i \geq \ell} v_i,$$

where v_i is the value on S_i and ℓ is the length of the counter prefix (which must start with the largest tile on S_0): a tile in the counter is worth its value discounted by its depth, every tile outside it is a liability of a thousand times its value.

At each step the generator considers three of the four directions—toward the head of the line being filled, toward the corner's own line, and away from the head—and, for the 2-fed line only and only when at most two cells are empty, the fourth, which the L-shaped assembly of the last line needs. Spawn cells are the first two empty snake cells and the cells one line deeper than the line being filled, where a tile can wait to drop in. The greedy step takes the best candidate if it raises H; otherwise a rescue search (iterative deepening to depth 10, ten best candidates per node, a fuel of 60,000 nodes) looks for a line that ends above the current H. Every candidate with at most two empty cells is checked five plies ahead for forced death. When no candidate and no rescue exists the position is a *stall*: its family (the tiles of value at least 8 and the number of empty cells) is banned, the walk retreats $8, 16, \ldots, 2048$ moves along its own line—replaying the line from the start restores the board—and continues around the ban. Mass grows every move, so the walk cannot cycle; a cap on the number of stalls bounds the work.

The fold. From the primed board the program searches the cascade of Lemma 5 by iterative deepening over total length: each move either advances the cascade by exactly the next doubling (a junk merge may ride along) or consolidates junk without touching the cascade, and the values spawned during the fold, 2 or 4, are chosen so that junk never lines up into an accidental merge that would displace the cascade. The primed board fills the board completely, so every junk spawn lands on the cascade's path; that the fold can nevertheless be done in the fifteen moves the lemma forces, with no consolidation moves at all, is a fact about the snake arrangement that the search establishes. For the tile line the fold is appended after the counter walk; in the full and score lines it happens mid-line, with one added rule—once the 131072 exists nothing may ever sit head-ward of it on the snake, since no smaller tile can merge its way past.

The 2-fed line. With 2s the counter needs one more cell per level (Lemma 7), and the second act, on fifteen cells beside the 131072, is one cell short of that at every turn. The generator opens a *relief valve* only under demonstrated necessity: after twelve stalls without progress (four near the end) the spawn menu admits 4s, every decision still exhausts its pure-2 options first, and the valve closes again after two hundred moves of progress. The two full-board squeezes—entering the fold and the last move—are handled by Proposition 15's logic in advance: there the 4 is granted outright.

6.3 What is proved about the output

Proposition 18 (certificate)**.** *Let a list of steps $(d_t, x_t, v_t)_{t=1}^{L}$ (direction, spawn cell, spawn value) be replayed from two starting tiles under the rules of the game. If every slide d_t changes the board*

and every cell x_t is empty when its spawn is placed, then the list is a play of the honest game with probability $\prod_t q_{v_t}/e_t > 0$, where e_t is the number of empty cells after slide t and $q_2 = 0.9$, $q_4 = 0.1$; its length, score and final position are then determined by Lemmas 2 and 3.

Theorem 19 (the shipped lines)**.** *Three lines pass the replay of Proposition 18, in each of the four corners and both orientations of Theorem 21:*

(a) 32,781 *moves from two 4s to a* 131072*, optimal by Theorem 4; its position after move* 32,766 *is the primed board of Lemma 5 arranged along the snake;*

(b) 65,533 *moves from two 4s to the chain along the snake, optimal by Corollary 6, scoring* 3,670,024*;*

(c) 129,333 *moves from two 2s to the chain along the snake with* $n_4 = 1{,}735$*, scoring* 3,925,224*—within* 6,876 *points of Corollary 10 and within* 1,719 *moves of Theorem 14.*

Proof. The replay is `test/gen_perfect.js` (all four corners, at generation time) and `PERFECT=1 node test/run.js` (through the game's own engine, either orientation); `test/paper_facts.js` checks the primed board and the last moves. The optimality claims are the theorems cited. □

6.4 The certificate, in a form anyone can check

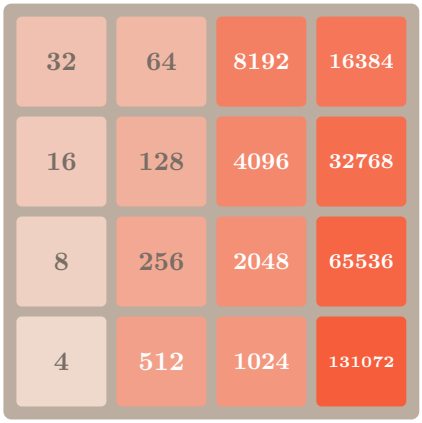
Proposition 18 is only as good as the replay, so the lines are also published in a form that owes nothing to this repository's code: three text files in `witness/` list the two starting tiles and then one line per move—the direction slid and the row, column and value of the tile that appeared—and `verify/verify2048.py`, 160 lines of Python with no dependencies, replays them under the rules of the original game. The verifier implements the slide twice, once cell by cell as in Cirulli's `game_manager.js` and once line by line, and stops at the first move where the two disagree, where a slide changes nothing, where a new tile is not a 2 or a 4, or where it lands on an occupied cell. On the three witnesses it reports, in under two seconds:

witness	steps verified	score	2s	4s	final position
`131072.txt`	32,781	1,966,092	8	32,775	131072 in the corner, 13 junk tiles
`full-chain.txt`	65,533	3,670,024	0	65,535	the full chain, dead
`max-score.txt`	129,333	3,925,224	127,600	1,735	the full chain, dead

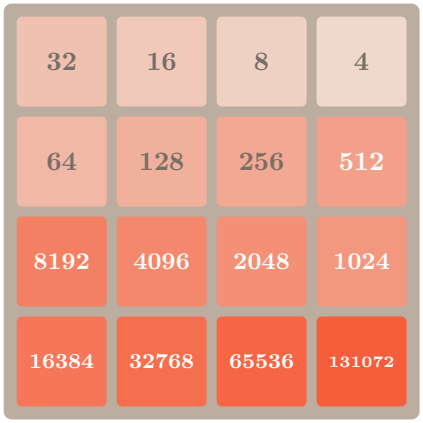
The counts of spawned 2s and 4s include the two starting tiles, and each report ends with "Illegal moves: 0, Illegal spawns: 0, Certificate: VALID". A second, deliberately different check (`test/replay_engine.js`) feeds the same text files to the original game engine—Cirulli's `game_manager.js`, `grid.js` and `tile.js`, unmodified, with the random tile replaced by the prescribed one—and confirms the same counts. Any implementation of the rules will do the same job in a few dozen lines; the format is documented in the files themselves. We are not aware of an earlier explicit, independently checkable certificate that the standard 4×4 game reaches 131072, let alone one of the minimum length.

6.5 Playing it

The application plays a line move by move. With planned spawns each move is exactly the step. Under honest spawns the player presses undo whenever the spawn differs from the plan and lets the game re-roll; each move is a run of geometric trials, counted in Section 8. Every position shown is a legal position of the game, and by Theorem 21 the same data serves all four corners and both directions out of each.



bottom-right corner, chain running up



bottom-right corner, chain running left

Figure 2: The final position of the perfect game: the full chain $131072, 65536, \ldots, 4$ along the two corner snakes out of the bottom-right corner. The left board is the application's default. Every cell holds a different power of two, so the position is dead; it is simultaneously the end of the 65,533-move line, the end of the 129,333-move maximum-score line, and by Corollary 13 the unique maximum-mass position of the game. The other six spirals are its images under the symmetries of the square (Theorem 21).

7 The eight spirals

The dihedral group D_4 of the square (four rotations, four reflections) acts on the cells of the board and on the four directions: a rotation by a quarter turn sends up to right, right to down, and so on; the reflection in a vertical axis swaps left and right and fixes up and down; and similarly for the rest.

Lemma 20 (equivariance)**.** *For every $g \in D_4$, position B and direction d,*

$$\mathrm{slide}_{gd}(gB) = g\,\mathrm{slide}_d(B),$$

with the same multiset of merged values. Consequently g maps every play to a play (transform each direction and each spawn cell) with the same length, the same score, the same spawned values, and the same number of empty cells after every slide.

Proof. A slide in direction d applies one fixed one-dimensional rule to each line parallel to d, the line read from the wall the tiles move toward. The symmetry g maps the lines parallel to d bijectively onto the lines parallel to gd and maps the wall d points to onto the wall gd points to, so it preserves the reading order of every line; the rule is the same, hence the identity, merge for merge. A spawn cell is empty before g is applied iff its image is empty after. □

Theorem 21 (eight spirals)**.** *(i) There are exactly eight corner snakes, and D_4 acts on them simply transitively: each is the image of any other under exactly one symmetry.*

(ii) If a play ends on the chain along the snake σ, then its image under $g \in D_4$ ends on the chain along $g\sigma$. Hence each shipped line yields eight plays with pairwise distinct final positions, of equal length, score and undo cost. In particular every one of the eight spiral arrangements of the full chain is reachable, each in exactly 65,533 *moves (the minimum) and each with* 3,925,224 *points by a* 129,333*-move play; and the* 131072 *can be built in any corner in* 32,781 *moves with the chain leaving the corner along either side.*

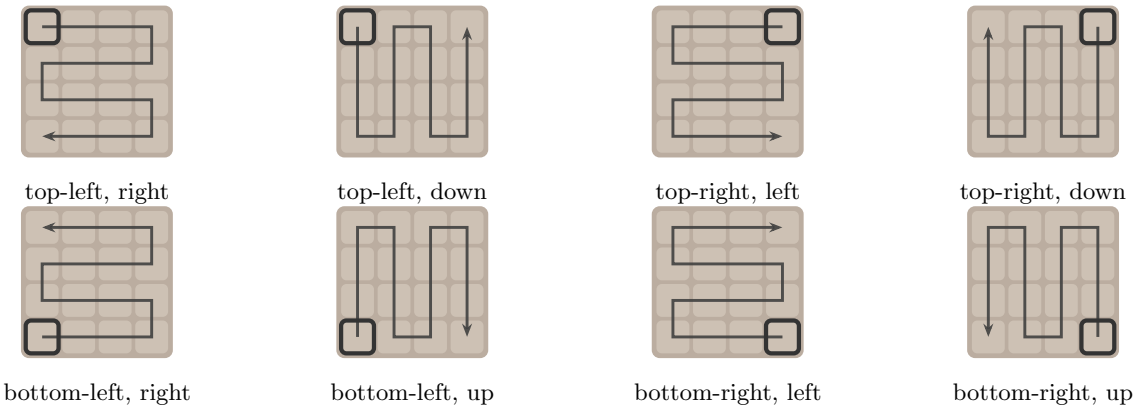


Figure 3: The eight corner snakes: one orbit of the symmetry group of the square. The marked cell is the corner that holds 131072; the arrow runs along the chain to the 4. The application exposes exactly this choice: a corner and a direction out of it.

(iii) Every corner snake ends at the corner adjacent to its start across the side perpendicular to its first direction, so the 4 sits at a corner with the 8 beside it in the snake's last line, as Proposition 15 requires.

Proof. (i) A corner snake is determined by its start corner and the side it first runs along, that is, by a *flag* (corner, incident side); there are $4 \cdot 2 = 8$ flags and the boustrophedon rule produces a Hamiltonian path from each. A symmetry that fixes a flag fixes two adjacent corners and is therefore the identity, so the stabiliser of a flag is trivial and the orbit of a flag under the group of order 8 has eight elements: all of them. (ii) is Lemma 20 applied to the whole play, together with Theorem 19 for the numbers; the eight final positions differ in the corner holding 131072 or in the cell holding 65536. (iii) The snake visits the four lines parallel to its first direction in turn, so its last cell is in the line farthest from the start, at the end nearest the start's column (an even number of lines are traversed after the first); that is the corner adjacent across the perpendicular side. □

Remark 22 (has this been shown before?). The snake in one corner is the standard human strategy and the standard target of heuristic players; the weight tables of Xiao's expectimax [34] and the priority map of Richardson's player [26] anchor one corner and one direction. Strategy guides describe one corner and one orientation. That the eight images are all attainable, in the same number of moves and by the same move list transformed, is an elementary consequence of the equivariance of the rules which we have not found stated in the literature; we record it because it is what makes one computed line eight perfect games, and because it is exactly the choice the application offers. Fifty-two Hamiltonian paths leave a corner of the 4×4 board, sixteen of them ending at a corner—always an adjacent one, since a Hamiltonian path on sixteen cells ends on the opposite colour of the checkerboard—and just two of the fifty-two are corner snakes. Which of the other arrangements of the chain, Hamiltonian or not, are reachable is open (Section 11); Proposition 15 rules out every arrangement with the 4 in the interior.

8 The undo button

8.1 The shipped lines

Three lines were generated once and are shipped as data: the 32,781-move line to 131072, the 65,533-move line to the full chain, and the 129,333-move maximum score line. The other three corners and the column-first spirals are the seven symmetries of the square applied to the same data (Theorem 21). A replay through the real engine checks that every slide moves, every spawn cell is empty, and the counts land exactly.

8.2 What a perfect game costs in undos

Proposition 23 (undo cost)**.** *Under honest spawns with a re-randomising undo, the expected number of undo presses needed to play a fixed line of L moves is*

$$\sum_{t=1}^{L}\Big(\frac{e_t}{q_{v_t}}-1\Big),$$

where e_t is the number of empty cells after slide t and $q_2 = 0.9$, $q_4 = 0.1$. A prescribed opening—values v_a, v_b in two prescribed cells of a c-cell board—needs an expected $c(c-1)/(2q_{v_a}q_{v_b}) - 1$ restarts.

Proof. The spawn after slide t lands in the required cell with the required value with probability $p_t = q_{v_t}/e_t$, independently at each attempt; the number of attempts is geometric with mean $1/p_t$, so the undos for the move are $1/p_t - 1$ in expectation, and expectations add. The opening places its two tiles in order, uniformly in c then $c-1$ cells, and both orders produce the prescribed pair. □

Two 4s in two prescribed cells of the 4×4 board happen one game in 12,000, so the line to 131072 starts with an expected 11,999 restarts. Summing the proposition over the shipped lines:

line	moves	𝔼[undos in play]	𝔼[opening restarts]	total
131072 (all 4s)	32,781	1,929,679	11,999	1,941,678
full chain (all 4s)	65,533	3,521,967	11,999	3,533,966
maximum score (2s)	129,333	658,984	147	659,131

Measured runs match: the browser reports 1.91–1.97 million undos for the 131072 line. The maximum-score line is four times longer but three times cheaper in undos, because a 2 is nine times as likely as a 4. By Lemma 20 the eight spirals cost the same.

9 Honest play

For the full board we implemented an expectimax player (Genius) in the tradition of Xiao's [34]: rows and columns are table lookups over ranks, the heuristic scores empty cells, available merges, monotonicity and a tile-sum penalty, with a pull toward the chosen corner; depth adapts to the number of distinct tiles. Against Richardson's [26] Evil generator, which places each new tile on the edge the last move packed against beside the largest neighbours, the search models that rule exactly instead of averaging over luck. We ported Richardson's four players (Smart, Algorithm, Priority, Random) and the generator to the same array engine for a like-for-like comparison. Ten games each, bottom-right corner:

question, standard 4×4 game	status
optimal move in every position; optimal expected score	open; 4×3 solved [11]; boards up to nine cells and the cube solved here (Table 2)
a strategy reaching 2048 with high probability	yes, probability ≥ 0.99969 [30]
a tile reachable with certainty	256 [30]; the optimal probability of each larger tile is open
largest tile	131072: bound folklore and [4], Corollary 12; reachable: the certificate of Section 6.4 (claimed for all boards by [4])
fewest moves to 2048	519, exact: [16] and Theorem 4
fewest moves to 131072	32,781, exact: Theorem 4 and the shipped line; every such win passes the primed board (Lemma 5)
maximum score	between 3,925,224 and 3,932,100 (Theorems 9, 19); conjectured 3,932,100
longest game	between 129,333 and 131,052 (Theorems 14, 19)
the full chain $131072, \ldots, 4$	reachable, in exactly 65,533 moves at the fewest (Corollary 6); eight spiral arrangements reachable (Theorem 21); every reachable arrangement has its 4 on the boundary beside the 8 (Proposition 15); which other arrangements are reachable is open
probability that an honest game reaches 131072	open; at least $10^{-57,047}$, the probability of the shipped line (Proposition 18)
number of reachable positions	open; more than $1.3 \cdot 10^{12}$ [14]

Table 3: What is known about the standard game, September 2026.

player	regular tiles	Evil tiles	regular, undo to escape death
Genius	16384 ×1, 8192 ×5, 4096 ×3, 1024 ×1; 130,131	4096 ×6, 2048 ×4; 56,850	32768 ×2 (2 games)
Smart	4096 ×3, 2048 ×6, 1024 ×1; 46,029	1024 ×4, 512 ×4, 256 ×2; 8,410	16384 ×3, 8192 ×2
Algorithm	512 ×5, 256 ×2, 128 ×3; 4,256	256/128/64; 2,011	1024 ×6, 512 ×4
Priority	512 ×1, 256 ×6, 128 ×2, 16 ×1; 2,952	128/64; 1,284	—
Random	128 ×6, 64 ×3, 32 ×1; 983	128/64/32; 508	256 ×9, 512 ×1

Semicolons separate the largest-tile histogram from the average score. The last column is the undo button used as a human uses it: only to escape game over, each death unwinding a doubling number of moves, giving up after forty deaths without a new best score.

10 Where the 4×4 game stands

Kaneko and Yamashita's strong solution of 4×3 [11] had to enumerate $1.15 \cdot 10^{12}$ positions for twelve cells. For the sixteen cells of the standard board nobody knows the count: Lees-Miller's combinatorial bound is about $4.4 \cdot 10^{16}$ positions for the game up to the 2048 tile and is known to be a large over-estimate [15], and his enumeration ran through $1.3 \cdot 10^{12}$ positions in a month without finishing [14]. *Strongly* solving 4×4—the optimal move in every reachable position and hence the optimal expected score and the optimal probability of every tile—is out of reach of this paper and, as far as we know, of any current computation; the layered pass of Section 5.5 is the right shape for it (it is Kaneko and Yamashita's), and the obstacle is purely size. What the previous sections settle is the other half of "perfect 2048", the possible-play half. Table 3 lists both halves.

Reaching the largest tile. The upper bound 2^{c+1} is elementary (Corollary 12); the question has always been attainability under the actual slide rules. Rodgers and Levine [27] called 131072 attainable "in theory" and the video evidence for it suspect; Lees-Miller [15] obtained it from

tile counting alone and left the geometric question open. Das and Paul [4] state attainability for every $n_1 \times n_2$ board as their Theorem 1. Their argument prescribes the spawns and proceeds by induction on the board, performing the construction for an $n_1 \times n_2$ board inside an $(n_1 + 1) \times n_2$ board and then using the extra cells; but a slide moves every line of the board, so a sub-board is not isolated from the rest, and the step needs an invariant that keeps the extra cells out of the way, which the paper does not state. We were unable to complete the argument from what is written, and we do not claim the general statement here. What is established: on every board up to nine cells, the cube included, by exhaustive enumeration (Table 1); on 4×3 by Kaneko and Yamashita's enumeration, which reaches $8192 = 2^{13}$; and on 4×4 by the certificate of Section 6.4, which also has the minimum possible length. A uniform construction with a proven invariant for all rectangles is open, and it is not the naive one: a snake counter that spawns at the far end of the frontier row and slides toward its head fails at the first turn of every board we tried, and our 4×4 line was found by search, not by a rule.

Predictions for 4×3. Kaneko and Yamashita's pass carries, per position, what the optimal policy needs. Adding a two-byte "fewest spawned 4s" field and a "fewest moves" field would decide the possible-play half of 4×3 at negligible cost, and this paper predicts the answers: the largest tile 8192 in $M(13) = 2{,}057$ moves and not fewer; the full chain $8192, \ldots, 4$ in $2^{12} - 3 = 4{,}093$ moves at the fewest and with exactly twelve spawned 4s at the fewest; a maximum score of $\Phi_{12} - 48 = 180{,}180$; a longest game of $2^{13} - 16 = 8{,}176$ moves; and $4{,}4 \to 8$ from a full board as the last move of every full-chain play. Every one of these predictions has held on every board we could solve.

Three dimensions. Das and Paul's game on a $2 \times 2 \times 2$ cube has eight cells, six directions and a symmetry group of order 48. Our solver handles boxes of any dimension (a slide is the same line rule along any axis), and the cube takes fourteen seconds: 702,581 positions up to symmetry, 32,187,234 raw. Every bound of this paper is attained on it: the largest tile $512 = 2^9$ is reached in exactly $M(9) = 133$ moves, the maximum score is $\Phi_8 - 32 = 7{,}140$, the longest game is $2^9 - 12 = 500$ moves, the full chain needs $253 = 2^8 - 3$ moves and eight spawned 4s, and all 9,216 chain-producing moves are $4 + 4 \to 8$ from a full board. In the honest game the cube is a better board than the 2×4 strip with the same eight cells: an optimal expected score of 2,953.42 against 2,641.87, a 256 tile with probability 94.04% against 86.98%, and a 512 with 6.32% against 2.46%. Six directions beat four.

11 Open problems

1. Strongly solving 4×4 (Table 3): the optimal expected score and the optimal probability of each tile, in particular of 131072, for which only the trivial lower bound of the shipped line is known.
2. Conjecture 16: a 4×4 play with sixteen spawned 4s, equivalently a 131,052-move game.
3. The maximum score and longest game of 4×3 (predicted 180,180 and 8,176) and 2×5 (predicted 36,828 and 2,034), within reach of layered enumeration.
4. A proof that 2^{c+1} is reachable on every rectangular board, i.e. a uniform strategy with a proven invariant (Das and Paul's Theorem 1); and then whether Theorems 4, 9 and 14 are tight on *every* board and in every dimension. The $2 \times 2 \times 3$ box (twelve cells) is the next three-dimensional case, at the scale of 4×3.

5. Which arrangements of the full chain are reachable on 4×4? At least the eight spirals are; every reachable one has its 4 on the boundary beside the 8 (Proposition 15). Can the 131072 sit off the corners, or the 4 at the corner opposite the 131072?
6. The boards passed at move $M(n) - (n-2)$ by the fastest plays to 2^n when 2^n is not the top tile (Lemma 5 fixes them only for $n = c + 1$).
7. Minimum undos: the expected undo count above is for the shipped lines; the line minimising expected undos (which would prefer spawns with many empty cells) is a different optimisation.

Reproducibility

All code is in the repository [32]. `node test/solve.js 3x3` reproduces Table 1 and Table 2 (3×3 takes about five minutes; `2x2x2` fourteen seconds); `PERFECT=1 node test/run.js` replays each shipped line through the game engine and asserts the exact counts (`GOAL=spiral|score`, `ORIENT=col`); `python3 verify/verify2048.py witness/131072.txt` checks a witness independently of everything else in the repository, and `node test/replay_engine.js witness/131072.txt` replays it through the original engine (`node test/gen_witness.js` regenerates the witnesses from the shipped data); `node test/paper_facts.js` checks Lemma 5, Proposition 15 and Theorem 21 against the shipped lines and, exhaustively, the tiny boards; `node test/gen_perfect.js` regenerates a line from nothing; `node test/honest.js` and `node test/honest_drive.js` produce the honest-play table. The board drawings are generated from the engine's own snake tables.

References

[1] Ahmed Abdelkader, Aditya Acharya, and Philip Dasler. 2048 without new tiles is still hard. In *8th International Conference on Fun with Algorithms (FUN 2016)*, volume 49 of *LIPIcs*, pages 1:1–1:14, 2016.

[2] Maggie Bai, Ava Kim Cohen, Eleanor Koss, and Charlie Lichtenbaum. Merge and conquer: Evolutionarily optimizing AI for 2048, 2025. arXiv:2510.20205.

[3] Gabriele Cirulli. 2048. `https://github.com/gabrielecirulli/2048`, 2014.

[4] Madhuparna Das and Goutam Paul. Analysis of the game "2048" and its generalization in higher dimensions, 2018. arXiv:1804.07393.

[5] David Eppstein. Making change in 2048. In *9th International Conference on Fun with Algorithms (FUN 2018)*, volume 100 of *LIPIcs*, pages 21:1–21:13, 2018. doi:10.4230/LIPIcs.FUN.2018.21.

[6] A. P. Ershov. On programming of arithmetic operations. *Communications of the ACM*, 1(8):3–6, 1958.

[7] Philippe Flajolet, Jean-Claude Raoult, and Jean Vuillemin. The number of registers required for evaluating arithmetic expressions. *Theoretical Computer Science*, 9(1):99–125, 1979.

[8] Hung Guei, Lung-Pin Chen, and I-Chen Wu. Optimistic temporal difference learning for 2048. arXiv:2111.11090, 2021.

[9] Hung Guei, Ting-Han Wei, and I-Chen Wu. 2048-like games for teaching reinforcement learning. *ICGA Journal*, 42, 2020. doi:10.3233/ICG-200144.

[10] Wojciech Jaśkowski. Mastering 2048 with delayed temporal coherence learning, multistage weight promotion, redundant encoding, and carousel shaping. *IEEE Transactions on Games*, 10(1):3–14, 2018.

[11] Tomoyuki Kaneko and Shuhei Yamashita. Strongly solving 2048 4×3. *ICGA Journal*, 2026. doi:10.1177/13896911261443437; arXiv:2510.04580.

[12] Iris Kohler, Theresa Migler, and Foaad Khosmood. Composition of basic heuristics for the game 2048. In *Proceedings of the 14th International Conference on the Foundations of Digital Games (FDG)*, 2019.

[13] Stefan Langerman and Yushi Uno. Threes!, Fives, 1024!, and 2048 are hard. In *8th International Conference on Fun with Algorithms (FUN 2016)*, volume 49 of *LIPIcs*, pages 22:1–22:14, 2016.

[14] John Lees-Miller. The mathematics of 2048: counting states by exhaustive enumeration. `https://jdlm.info/articles/2017/12/10/counting-states-enumeration-2048.html`, 2017.

[15] John Lees-Miller. The mathematics of 2048: Counting states with combinatorics. `https://jdlm.info/articles/2017/09/17/counting-states-combinatorics-2048.html`, 2017.

[16] John Lees-Miller. The mathematics of 2048: minimum moves to win with Markov chains. `https://jdlm.info/articles/2017/08/05/markov-chain-2048.html`, 2017.

[17] John Lees-Miller. The mathematics of 2048: optimal play with Markov decision processes. `https://jdlm.info/articles/2018/03/18/markov-decision-process-2048.html`, 2018.

[18] Kiminori Matsuzaki. Systematic selection of N-tuple networks with consideration of interinfluence for game 2048. In *Technologies and Applications of Artificial Intelligence (TAAI 2016)*. IEEE, 2016.

[19] Kiminori Matsuzaki. Developing a 2048 player with backward temporal coherence learning and restart. In *Advances in Computer Games (ACG 2017)*, pages 176–187. Springer, 2017.

[20] Kiminori Matsuzaki and Madoka Teramura. A further investigation of neural network players for game 2048. In *Advances in Computer Games (ACG 2019)*. Springer, 2019.

[21] Rahul Mehta. 2048 is (PSPACE) hard, but sometimes easy. arXiv:1408.6315, 2014.

[22] Alok Menghrajani. 2048 with undo. `https://github.com/alokmenghrajani/2048`, 2014.

[23] Kazuto Oka and Kiminori Matsuzaki. Systematic selection of N-tuple networks for 2048. In *Computers and Games (CG 2016)*. Springer, 2016.

[24] Michael S. Paterson and Carl E. Hewitt. Comparative schematology. In *Record of the Project MAC Conference on Concurrent Systems and Parallel Computation*, pages 119–127, 1970.

[25] ProbabilitySports. Solving the 3×3 variant of 2048. `https://probabilitysports.com/2048.html`. Accessed September 2026.

[26] A. J. Richardson. 2048-AI. `https://github.com/aj-r/2048-AI`, 2014.

[27] Philip Rodgers and John Levine. An investigation into 2048 AI strategies. In *IEEE Conference on Computational Intelligence and Games (CIG)*, 2014.

[28] Prady Saligram, Tanvir Bhathal, and Robby Manihani. 2048: Reinforcement learning in a delayed reward environment, 2025. arXiv:2507.05465; withdrawn by the authors pending regenerated results.

[29] Ravi Sethi and Jeffrey D. Ullman. The generation of optimal code for arithmetic expressions. *Journal of the ACM*, 17(4):715–728, 1970.

[30] Alexey Slizkov. Computational bounds for the 2048 game. arXiv:2303.07266, 2023.

[31] Marcin Szubert and Wojciech Jaśkowski. Temporal difference learning of N-tuple networks for the game 2048. In *IEEE Conference on Computational Intelligence and Games (CIG)*, pages 1–8, 2014.

[32] Dom the Developer. 2048 Superintelligence. `https://github.com/DomTheDeveloper/2048-undo`, 2026.

[33] I-Chen Wu, Kun-Hao Yeh, Chao-Chin Liang, Chia-Chuan Chang, and Han Chiang. Multi-stage temporal difference learning for 2048. In *Technologies and Applications of Artificial Intelligence (TAAI 2014)*, Lecture Notes in Computer Science. Springer, 2014. doi:10.1007/978-3-319-13987-6_34.

[34] Robert Xiao. 2048-ai: an expectimax AI for 2048. `https://github.com/nneonneo/2048-ai`, 2014.

[35] Shuhei Yamashita, Tomoyuki Kaneko, and Tomoyuki Nakayashiki. Strongly solving 2048 on 3×3 board and performance evaluation of reinforcement learning agents. In *Game Programming Workshop*. IPSJ, 2022. In Japanese.